Project Technical Poster

Project Overview

This Roblox horror/survival project is built around player movement, monster AI, stage progression, interactive tasks, tools, and client/server UI updates.

The project uses Roblox Lua/Luau ModuleScripts as class-like objects. Important systems include:

- `game.ReplicatedStorage.ClientClasses.Player`: client player controller composed from movement, camera, door, flashlight, and tool modules.
- `game.ServerStorage.Classes.MonsterManager`: server-side monster factory and AI class system.
- `game.ServerStorage.Classes.TaskManager`: server-side task assignment, ownership, progress tracking, and task UI replication.
- `game.ServerStorage.Classes.StageManager`: stage progression controller.
- `game.ReplicatedStorage.Packets.BytePackets` and `game.ReplicatedStorage.Remotes`: client/server communication.

OOP Implementation Summary

Roblox Lua/Luau does not have built-in `class`, `private`, `public`, `protected`, `abstract`, `interface`, or `template` keywords.

This project represents OOP with:

- ModuleScripts that return tables.
- `__index` metatables for method lookup.
- Constructor functions such as `.new()`.
- Colon syntax such as `self:Start()` and `self:Complete(player)`.
- Server-only modules for authority and data protection.
- RemoteEvents and BytePackets for controlled client/server communication.
- Base modules such as `BaseMonster`, `BaseTask`, and `BaseTool`.

Four Pillars of OOP in This Project

Encapsulation

Definition: Keeps object data and implementation details inside a controlled boundary.

Lua/Roblox replacement: `local` variables/functions inside modules, selected returned APIs, server-only scripts, and server validation.

Example from code: `game.ServerStorage.Classes.TaskManager`

- Local state is hidden inside the module: `tasks`, `taskById`, `assignedTasksByPlayer`, `assignedTaskIds`, `connections`.

- Only selected methods are returned: `init()`, `getPlayerTasks()`, `assignTasks()`, `getTaskById()`, `completeTask()`, `destroy()`.
- Clients cannot directly assign or complete tasks. Server task classes validate ownership through `BaseTask:IsAssignedTo(player)`.
- UI receives only simplified rows through `TaskUpdate`, not raw server task objects.

Why it matters: Task ownership and completion are protected from client-side tampering.

Abstraction

Definition: Hides complex behavior behind simple methods.

Lua/Roblox replacement: Public methods on ModuleScript tables, manager methods, and event-driven APIs.

Example from code: `game.ServerStorage.Classes.MonsterManager.BaseMonster`

- `BaseMonster:Start()` hides setup details by calling `_connectBaseEvents()`, optional `_connectEvents()`, and `_startAI()`.
- `BaseMonster:_update()` hides AI decision logic for looking, pathing, direct movement, and stopping.
- `MonsterManager.Summon("Bonnie")` hides cloning and class construction.

Why it matters: Other systems can spawn or start monsters without knowing every AI detail.

Inheritance

Definition: A class reuses and extends behavior from a parent/base class.

Lua/Roblox replacement: Metatable inheritance through `setmetatable(subclass, {__index = baseClass})`.

Example from code:`game.ServerStorage.Classes.MonsterManager.BaseMonster`

- `BaseMonster:Extend(className)` creates subclasses.
- `Mannequin = BaseMonster:Extend("Mannequin")`.
- `Bonnie = BaseMonster:Extend("Bonnie")`.
- Subclasses reuse base movement, line-of-sight, cleanup, and event wiring, then add custom behavior.

Another Example: `Game.ServerStorage.Classes.TaskManager.Tasks.BaseTask`

- `PromptTask`, `Valve`, and `DrinkWater` are created with `setmetatable({}, BaseTask)`.
- They all share assignment, prompt enabling, completion, attributes, and update events.

Why it matters: Shared logic is written once, while each monster/task keeps its own special behavior.

Polymorphism

Definition: Different object types expose the same method names but perform different behavior.

Lua/Roblox replacement: Duck typing: if an object has `:Start()`, `:Complete()`, `:Destroy()`, or `:onEquipped()`, managers can call it without knowing its exact class.

Example from code: `TaskManager.Tasks`

- `PromptTask:Start()` completes when a prompt is triggered.
- `Valve:Start()` listens for hold-began/hold-ended and rotates a valve until target progress is reached.
- `DrinkWater:Start()` triggers a drinking sequence.
- `TaskManager` simply calls `taskObject:Start()` and later `taskObject:Destroy()`.

Another example: `toolHandler`

- `toolHandler` creates a tool object and calls `onEquipped`, `onUnequipped`, and `onActivated`.
- `BaseTool` provides default behavior.
- `Broom` inherits from `BaseTool` and can override behavior later.

Why it matters: New task or tool types can be added with the same method contract without rewriting the manager.

OOP Feature Mapping: C++ vs Roblox Lua/Luau

C++ feature	Roblox Lua/Luau equivalent in this project
`private` / `public` / `protected`	`local` variables/functions, returned module methods, server/client boundaries
Classes	ModuleScripts returning tables with `__index`
Constructors	`.new()` functions, usually using `setmetatable({}, Class)`
Inheritance	`BaseMonster:Extend()`, `BaseTool:Extend()`, and `setmetatable(subclass, {__index = Base})`

Interfaces	Not formally implemented; replaced by method contracts such as `:Start()`, `:Destroy()`, `:Complete()`, `:onEquipped()`
Abstract classes	Base modules such as `BaseTask`, `BaseTool`, and `BaseMonster`; no formal abstract keyword
Templates / generics	Reusable ModuleScripts, factory functions, config tables, and Luau type annotations where used
Polymorphism	Managers call shared method names on different module objects; Lua duck typing

Use Case Diagram

```mermaid
flowchart LR
    Player((Player))
    Client[Client Scripts]
    Server[Server Game System]
    MonsterSystem[MonsterManager]
    TaskSystem[TaskManager]
    StageSystem[StageManager]
    UISystem[Tasks UI]

    Player --> Move[Move / sprint / crouch]
    Move --> Client
    Client --> Movement[Player.movement]

    Player --> UsePrompt[Trigger task ProximityPrompt]
    UsePrompt --> Server
    Server --> TaskSystem
    TaskSystem --> UISystem

    Player --> LookAtMonster[Look at monster]
    LookAtMonster --> Client
    Client --> Server
    Server --> MonsterSystem

    Player --> PullLever[Pull generator lever]
    PullLever --> StageSystem
    StageSystem --> TaskSystem

    Player --> EquipTool[Equip/use Broom tool]
    EquipTool --> Client
    Client --> ToolHandler[Player.toolHandler]
```

Class Diagram

```mermaid
classDiagram
    class PlayerClass {
        +Player plr
        +Model Character
        +Humanoid Humanoid
        +BasePart Root
        +ToolHandler ToolHandler
        +new(character)
        +connectEvents()
        +initInputs()
        +startLoops()
        +Destroy()
    }

    class Movement {
        +Vector3 velocity
        +BasePart collider
        +boolean Crouching
        +new(playerClass)
        +update(dt)
        +toggleCrouch()
        +Destroy()
    }

    class ToolHandler {
        +Backpack Backpack
        +ToolModules ToolModules
        +ToolInstances ToolInstances
        +new(playerClass)
        +connectEvents()
        +Destroy()
    }

    class BaseTool {
        +Tool Tool
        +BasePart Handle
        +AnimationTracks AnimationTracks
        +Extend(className)
        +new(class, playerClass, tool)
        +onEquipped()
        +onUnequipped()
        +onActivated()
        +Destroy()
    }

    class Broom {
```

```
    +Motor6D Joint
    +new(playerClass, tool)
    +onEquipped()
    +onUnequipped()
    +onActivated()
    +Destroy()
}

class MonsterManager {
    +Summon(MonsterName)
}

class BaseMonster {
    +Model Model
    +BasePart Root
    +Humanoid Humanoid
    +boolean Angry
    +Extend(className)
    +new(class, model)
    +Start()
    +Destroy()
    +_update()
    +_moveDirect(position)
    +_computePath(targetPosition)
}

class Mannequin {
    +Anims Anims
    +new(model)
    +_connectEvents()
    +_handleCatch(hit)
}

class Bonnie {
    +State State
    +ChasingTarget ChasingTarget
    +new(model)
    +Destroy()
    +_connectEvents()
    +_update()
}

class TaskManager {
    +init()
    +assignTasks(player)
    +getPlayerTasks(player)
    +completeTask(taskId, player)
```

```
    +destroy()
}

class BaseTask {
    +Id Id
    +Type Type
    +Category Category
    +Prompt Prompt
    +AssignedPlayer AssignedPlayer
    +Start()
    +Assign(player)
    +Unassign()
    +Complete(player)
    +Destroy()
}

class PromptTask {
    +Start()
}

class Valve {
    +ValvePart ValvePart
    +TargetRotation TargetRotation
    +CurrentRotation CurrentRotation
    +Start()
    +Destroy()
}

class DrinkWater {
    +WaterCup WaterCup
    +Start()
    +_drink(player)
}

PlayerClass *-- Movement
PlayerClass *-- ToolHandler
ToolHandler o-- BaseTool
BaseTool <|-- Broom

MonsterManager o-- BaseMonster
BaseMonster <|-- Mannequin
BaseMonster <|-- Bonnie

TaskManager o-- BaseTask
BaseTask <|-- PromptTask
BaseTask <|-- Valve
BaseTask <|-- DrinkWater
```

```

## Sequence Diagram

Critical process: player completes an assigned task and the UI updates.

```mermaid
sequenceDiagram
    actor Player
    participant Prompt as ProximityPrompt
    participant Task as Task Object
    participant BaseTask
    participant TM as TaskManager
    participant Remote as TaskUpdate RemoteEvent
    participant Client as eventHandler
    participant UI as Tasks ScreenGui

    Player->>Prompt: Trigger or hold prompt
    Prompt->>Task: Start-specific event handler
    Task->>BaseTask: Complete(player)
    BaseTask->>BaseTask: Validate IsAssignedTo(player)
    BaseTask->>BaseTask: Set Completed and disable prompt
    BaseTask-->>TM: Updated BindableEvent fires
    TM->>TM: Build grouped rows
    TM->>Remote: FireClient(player, rows)
    Remote-->>Client: OnClientEvent(rows)
    Client->>UI: Clone/update template rows
    UI-->>Player: Show progress like (1/2)
```

Critical Process Explanation

When a player interacts with a task prompt, the specific task class handles the behavior.

- `PromptTask` completes immediately after a valid trigger.
- `Valve` tracks hold progress and rotates the physical valve.
- `DrinkWater` seats the player, welds the cup near the head, plays the drink sound, then completes.

All task classes call `BaseTask:Complete(player)`, which checks that the player is actually assigned to that task. When completion succeeds, `BaseTask` fires `Updated`. `TaskManager` catches that event, groups the player's tasks by type, and sends a small UI payload to `StarterPlayerScripts.eventHandler`, which updates `StarterGui.Tasks`.

Conclusion
```

This project demonstrates OOP in Roblox Lua/Luau through ModuleScript classes, constructor functions, metatable inheritance, composition, and polymorphic method contracts.

The strongest OOP examples are:

- `BaseMonster` with `Mannequin` and `Bonnie`.
- `BaseTask` with `PromptTask`, `Valve`, and `DrinkWater`.
- `BaseTool` with `Broom`.
- `PlayerClass` composing movement, camera, door, flashlight, and tool systems.

Not currently implemented: formal interfaces or abstract errors. A realistic improvement would be adding base methods such as `BaseTask:Start()` that warn or error if a subclass forgets to override them.